

GEI1072

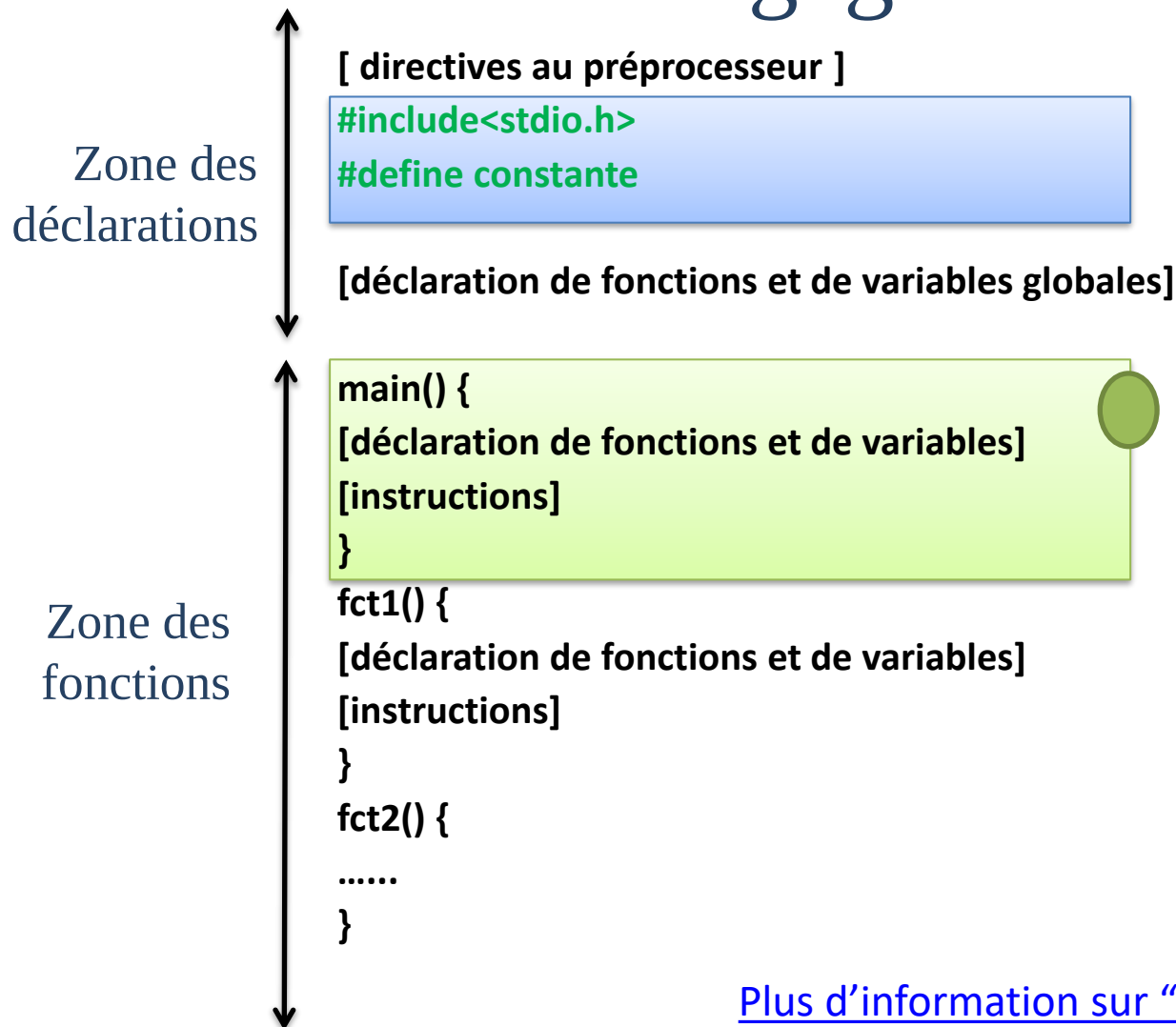
Résolution de problèmes d'ingénierie en C

Miloud Bagaa

miloud.bagaa@uqtr.ca



Structure de base d'un programme en langage C



[Plus d'information sur "include" et la fonction main.](#)

Variables et constantes

Déclarations des variables

```
type nom_de_la_variable [= valeur];
```

```
int X = 5;
```

[Plus d'information sur les variables.](#)

➤ nom_de_la_variable

- Unique pour chaque variable (selon leur portée)
- Commence toujours par une lettre
- Différenciation minuscule-majuscule (**i.e., case sensitive**)

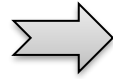
➤ type

- Conditionne le format de la variable en mémoire
- Peut être soit un type standard ou un type utilisateur

➤ valeur

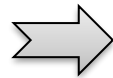
- Peut évoluer pendant l'exécution
- initialisation grâce à l'opérateur d'affectation

Nomination des variables



- Un nom significatif.
- La même norme utilisée dans l'équipe.

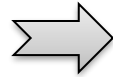
camelCase



userLoginCount

Variables et fonctions

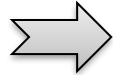
PascalCase



UserLoginCount

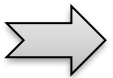
Classes

snake_case



user_login_count

kebab-case



User-login-count

[Plus d'information.](#)

Caractéristiques du C

➤ C est un langage typé

- Toute variable, constante ou fonction est d'un type précis.

➤ Les types de bases de C

- char : particularité du type caractère en C'est qu'il est assimilé à un entier (le code ASCII).
- int.
- float, double.
- short, long, unsigned.

[Plus d'information sur le code ASCII.](#)

[Plus d'information sur: %c](#)

```
#include<stdio.h>
```

```
void main() {  
    char x = 'D';  
    printf("The lowercase character of x is %c \n", x  
- 'A' + 'a');  
}
```

Ou voir le slide 34.

Les types entiers

- int : mot naturel de la machine. Pour PC Intel = 32bits.
- int peut être précédé par
 - un attribut de précision : **short** ou **long**
 - un attribut de représentation : **unsigned**

Caractère	char	8 bits
Entier Court	short	16 bits
Entier	int	32 bits
Entier Long	long	32 bits

[Plus d'information sur les variables.](#)

[Plus d'information sur les types.](#)

- Librairie standard limits.h comporte les définitions et caractéristiques des types communs.
 - https://www.tutorialspoint.com/c_standard_library/limits_h.htm

Les types flottants

- Les types **float**, **double** et **long double** représentent les nombres en virgule flottante.

Flottant	float	32 bits
Flottant double précision	double	64 bits
Flottant quadruple précision	long double	128 bits

- Représentation normalisée
 - signe, mantisse 2^{exposant}

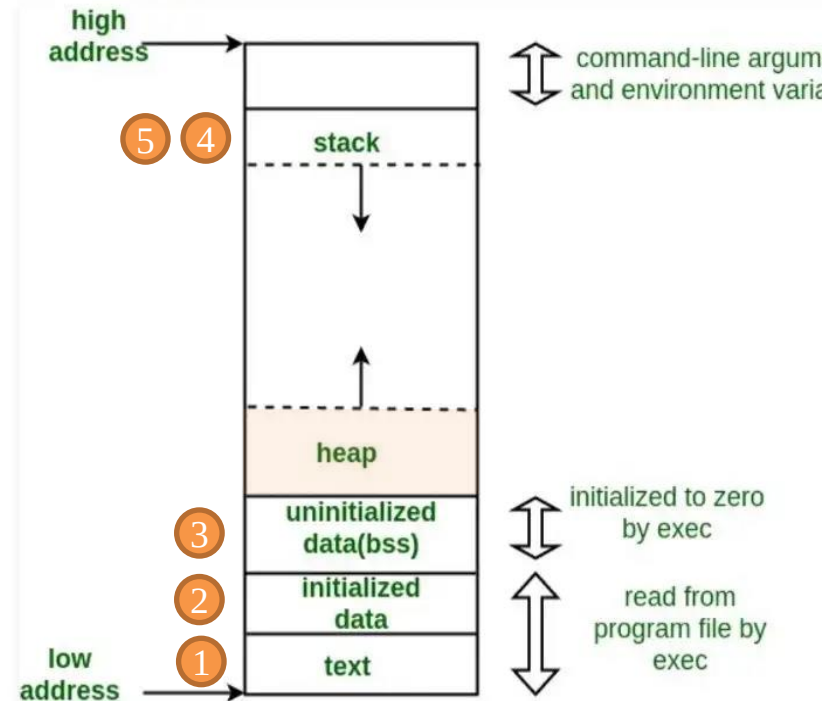
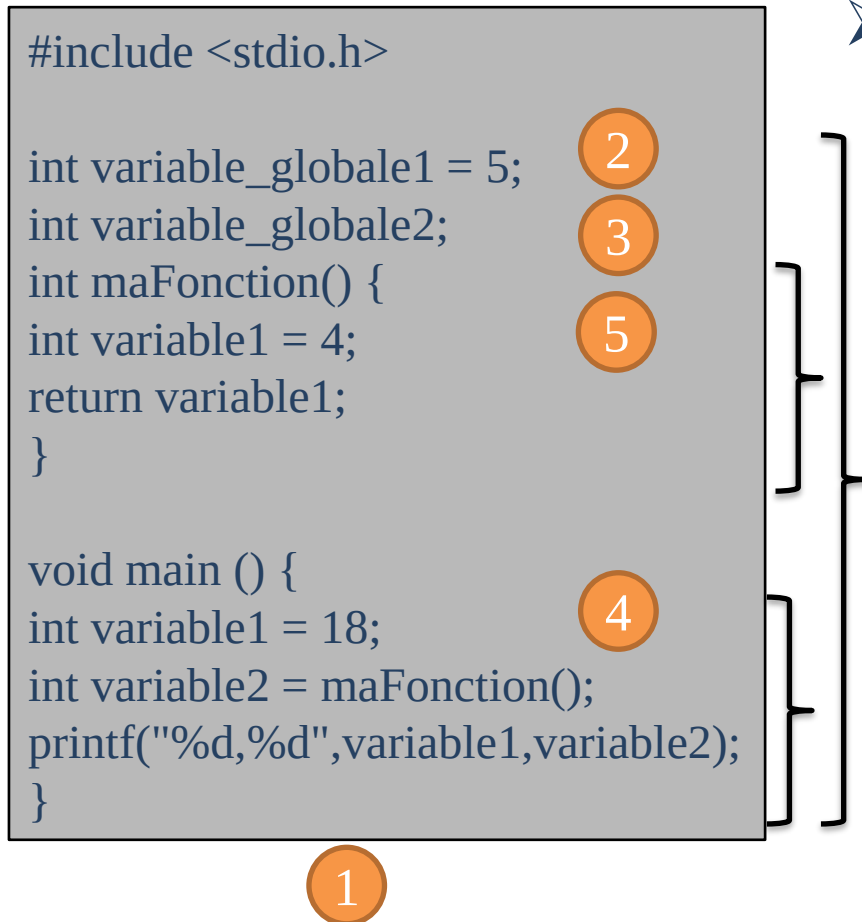
Portée des variables

➤ Les variables **locales**

- définies dans une fonction ou dans un bloc
- leur portée est alors la fonction ou le bloc

➤ Les variables **globales**

- définies hors des fonctions
- leur portée est **TOUT** le programme



<https://www.geeksforgeeks.org/stack-vs-heap-memory-allocation/>

Les constantes

- Valeur qui apparaît littéralement dans le code source
 - Le type de constante est déterminé par la façon dont la constante est écrite.
 - 42 est une constante entière
 - 3.14 est une constante flottante

```
const type nom_de_la_constant = valeur;
```

```
const int X = 5;
```

Les constantes pré-processeur

- Permet de donner un nom symbolique a une constante

```
#define nom_de_la_constant valeur
```

```
#define my_constant 5
```

- À la compilation, le préprocesseur change toutes les occurrences de **nom_de_la_constant** en **valeur**

Les constantes caractères

- Exception des caractères imprimables \ ' ? et "
 - Ils sont désignés par \\ \' \? et \«
- Caractères non-imprimables
 - peuvent être désignés par '`code-octal`' ou '`xcode-hexa`', par exemple : '`33`' ou '`x1b`' désigne le caractère '`espace`'.

\n nouvelle ligne

\t tabulation horizontale

\v tabulation verticale

\b retour arrière

\r retour chariot

\f saut de page

\a signal d'alerte

Les constantes : exemple

```
#include <stdio.h>

#define PI 3.14 //constante préprocesseur

float circonference(int rayon){

    return (2*PI*rayon);
}

void main () {
    float rayon, resultat;

    const char* MESSAGE = "La circonférence est : "; //variable constante

    printf ("Entrez le rayon\r\n"); //constantes chaînes et caractères

    scanf("%f", &rayon);

    resultat = circonference(rayon);

    printf("%s \v %f \n", MESSAGE, resultat);

}
```

Assignations (Affectation)

variable = expression;

- **expression** est évaluée et est affectée à **variable**
- L'affectation effectue une conversion de type implicite
 - la valeur de **expression** est convertit dans le type de **variable**

[Plus d'information sur: %d %f](#)

Ou voir le slide 34.

```
#include <stdio.h>

void main () {

int variable1, variable2 = 2;

float variable3 = 2.5 ;

variable1 = variable1 + variable2;

variable3 = variable3 + variable2;

printf("\n variable1 = %d ***
variable3 = %f \n", variable1,
variable3);

}
```

Les Opérateurs

© GEI1072 – Miloud Bagaa

Les opérations arithmétiques

- **Opérateur unaire** : –
- **Opérateurs binaires** : +, -, *, /, % (reste de la division = modulo)
- **La division est désignée par /**
 - Si les 2 opérandes sont de types entier, / produira une division entière

```
float x;  
x = 3/2; /*affecte à x la valeur 1*/  
x = 3/2.0; /*affecte à x la valeur 1,5*/  
x = 2 * 3 + 5;  
x = 2 * (3 + 5);
```

[Plus d'information sur les opérations.](#)

Les opérateurs relationnels

➤ La syntaxe est:

opération1 **op** opération2;

> strictement supérieur

>= supérieur ou égal

< strictement inférieur

<= inférieur ou égal

== égal

!= différent

➤ La valeur booléenne rendue est de type int

1 pour **vrai**

0 **sinon**

```
#include<stdio.h>

void main() {
    int y = 3, z = 3;
    printf(" y == z : %d \n", y == z);
}
```

Les opérateurs logiques

➤ Booléens :

&& et logique

|| ou logique

! négation logique

➤ Bit à bit :

& et logique

| ou inclusif

^ ou exclusif

~ complément à 1

<< décalage à gauche

>> décalage à droite

Opérateurs d'affectation composée

`+=` `-=` `*=` `/=` `&=` `^=` `|=` `<<=` `>>=`

➤ Pour tout opérateur **op** :

opération1 **op**= opération2;

`X += 5;`



opération1 = opération1 **op** opération2;

`X = X + 5;`

➤ **opération1** n'est évalué qu'une seule fois

Opérateurs incrémentation ou décrémentation

- Incrémentation : **++**
- Décrémentation : **--**
- En suffixe **i++** ou en préfixe **++i** :
 - dans les deux cas la valeur de i sera **incrémentée**
 - pour le premier cas la valeur **affectée est l'ancienne**

```
int a = 3, b, c;  
b = ++a; /*a et b valent 4*/  
c = b++; /*c vaut 4 et b valent 5*/  
}
```

Les structures de contrôle

Branchement conditionnel

if (expression1)

 instructions;

else

 instructions;

Le else est facultatif

if (expression1)

 instructions;

[Plus d'information sur les conditions.](#)

```
void main()
{
    int x = 4;
    int y = 0;
    if (x >= 3)
        y = 5;
    else
        y = 6;
    printf("y = %d \n",y);
}
```

```
if(1) {
    printf("The value is vrai\n");
}
```

Branchement conditionnel

expression est évaluée. Si le résultat vaut **valeur1**, alors **instruction1** est exécutée et on quitte le switch, sinon si **expression** vaut **valeur2**,....., **sinon** on va dans **default** pour exécuter **instructionD**.

```
int y =3;
switch (y)
{
    case 3:
        printf("The value of y is three\n");
        break;
    case 4:
        printf("The value of y is four\n");
        break;
    default:
        printf("The value of y is different then three and
four\n");
        break;
}
```

```
switch (expression)
{
    case valeur1:
        instruction1;
        break;
    case valeur2:
        instruction2;
        break;
    .
    .
    case valeurN:
        instructionN;
        break;
    default:
        instructionD;
        break;
}
```

Les boucles

➤ La boucle **for**

```
for (expression1; expression2; expression3) {  
    instruction1;  
    .  
    .  
    instructionN;  
}
```

```
int i = 1;  
for (i = 0; i < 10; i++) {  
    printf("\n i = %d",i);  
}
```

Affiche les entiers de 1 à 9

À la fin de la boucle i vaut 10

[Plus d'information sur les boucles.](#)

Les boucles

➤ La boucle **while**

```
while (expression)
{
    instruction1;
    .
    .
    instructionN;
}
```

- Tant que **expression** est vrai, le bloc d'instruction est exécuté.
- Si **expression** est faux, le bloc d'instruction n'est pas exécuté.

```
int i = 1;
while (i < 10) {
    printf("\n i = %d",i);
    i++;
}
```

Affiche les entiers de 1 à 9

[Plus d'information sur les boucles.](#)

Les boucles

➤ La boucle **do while**

```
do (expression)
{
    instruction1;
    .
    .
    instructionN;
}
while (expression)
```

- Le bloc d'instruction est exécuté tant que **expression** est vrai.
- Le bloc d'instruction est toujours exécuté au moins une fois.

```
int a;
do {
    printf("Entrez entier entre 1 et 10 :\n");
    scanf("%d",&a);
} while ((a <= 0) || (a >= 10))
```

[Plus d'information sur les boucles.](#)

Branchement non conditionnel

- **goto**
- Permet de se brancher à un emplacement quelconque du programme dans la même fonction.

goto est à éviter

```
main() {  
    int i;  
    for (i = 0; i < 5; i++) {  
        printf("boucle for : ")  
        if (i == 2)  
            goto sortie;  
        printf("%d fois \n", i+1);  
    }  
    sortie : printf("\n Fin i = %d\n", i);  
}
```

donne comme résultat :
boucle for 1 fois
boucle for 2 fois
boucle for
fin

Branchement non conditionnel

- **continue**
- Permet de **passer** directement à **l'itération suivante** sans exécuter les autres instructions de la boucle.

```
main() {  
    int i;  
    for (i = 0; i < 5; i++) {  
        if (i == 3)  
            continue;  
        printf("i = %d\n", i);  
    }  
    printf("valeur de i en fin de boucle = %d\n", i);  
}  
---  
imprime i = 0, i = 1, i = 2, i = 4  
valeur de i en fin de boucle = 5
```

[Plus d'information sur continue.](#)

Opérateur virgule

- Suite d'expressions séparées par une virgule

expression1, expression2, expression3

- Expression évaluée de gauche à droite

```
main() {  
    int a, b;  
    b = ((a = 3) , (a + 2));  
    printf(" b = %d\n", b);  
}  
---  
imprime b = 5
```

Opérateur conditionnel ternaire

- Opérateur conditionnel : ?

Condition ? expression1 : expression2

- Le résultat est égal à **expression1** si la condition est **vraie** et à **expression2** sinon

```
x >= 0 ? x : -x; /*correspond à la valeur absolue*/  
m = ((a < b) ? a : b); /*affecte à m le maximum de a et b*/
```

Opérateur de conversion de type

- Appellé **cast**
- Permet de modifier explicitement le type d'un objet

(type) objet

```
main()
{
  int i = 3, j = 2;
  printf("i/j =%f\n", (float) i/j);
}
```

Renvoie la valeur 1.5

Opérateur adresse

- L'opérateur d'adresse & appliqué à une variable retourne l'adresse-mémoire de cette valeur

&objet

```
main()
{
  int x = 4;
  int *y = &x;
}
```

Mémoire du programme

		Address
		&x
x	4	0x3dc
y	0x3dc	0x3d8
	NA	0x3d4
	NA	0x3d0

Les fonctions d'entrées/sorties classiques

- Fonctions de la librairie standard `stdio.h` : clavier et écran, appel par la directive

`#include<stdio.h>`

- Cette directive n'est (souvent) pas nécessaire pour `printf` et `scanf`.
- Fonction d'écriture `printf` permet une impression formatée

```
printf("chaîne de contrôle", expr1, ..., exprn);
```

- Chaîne de contrôle spécifie le texte à afficher et les formats correspondant à chaque expression de la liste.
- Les formats sont introduits par `%` suivi d'un caractère désignant le format d'impression.

Les fonctions d'entrées/sorties classiques

format	conversion en	écriture
%d	int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	unsigned int	octale non signée
%lo	unsigned long int	octale non signée
%x	unsigned int	hexadécimale non signée
%lx	unsigned long int	hexadécimale non signée
%f	double	décimale virgule fixe
%lf	long double	décimale virgule fixe
%e	double	décimale notation exponentielle
%le	long double	décimale notation exponentielle
%g	double	décimale, représentation la plus courte parmi %f et %e
%lg	long double	décimale, représentation la plus courte parmi %lf et %le
%c	unsigned char	caractère
%s	char*	chaîne de caractères

Les fonctions d'entrées/sorties classiques

- En plus, entre le % et le caractère de format, on peut éventuellement préciser certains paramètres du format d'impression
 - %10d : au moins 10 caractères réservés pour imprimer l'entier.
 - %10.2f : on réserve 12 caractères (en plus du .) pour imprimer le flottant avec 2 chiffres après la virgule.
 - %30.4s : on réserve 30 caractères pour imprimer la chaîne de caractères, mais que 4 seulement seront imprimés suivis de 26 blancs.

Les fonctions d'entrées/sorties classiques

➤ Fonction de saisie des données au clavier **scanf**

```
scanf("chaîne de contrôle", arg1,arg2 ..., argn);
```

- Chaîne de contrôle indique le format dans lequel les données lues sont converties.
- Ne contient pas le caractère "\n".
- Même format que **printf** avec une légère différence.

```
main() {  
    int i;  
    printf("entrez un entier sous forme hexadécimale i =");  
    scanf("%x", &i);  
    printf("i = %d\n",i);  
}
```

Si la valeur 1a est saisie alors le programme affiche i = 26

Merci!